

Level 1 – Understanding

Q1.A query joins branch, customer, account, account_txn and card. How many join conditions does it need, and what happens if you supply one fewer?

Answer: There are 4 join conditions because 5 tables are connected. If one is missing, the tables may not be properly connected and the result can contain incorrect rows.

Q2.Explain the difference between the two joins and state which returns more rows.

Answer: INNER JOIN returns only customers who have an account. LEFT JOIN returns all customers, including those without an account. So LEFT JOIN can return more rows.

Q3.The overdue-loan query returns only branches that have overdue loans. Explain the defect and write the fix.

Answer: The WHERE condition removes branches where the loan is NULL. Put the status condition inside the LEFT JOIN.

```
SELECT b.branch_name, l.loan_id
FROM branch b
LEFT JOIN loan l
ON l.branch_id = b.branch_id
AND l.status = 'OVERDUE';
```

Q4. Write the general shape of an anti-join and state which column must be tested for NULL.

Answer: Use LEFT JOIN and check a non-nullable column from the right table for NULL. Usually the primary key is used because NULL means no matching row was found.

Q5. Why does the employee hierarchy query need a LEFT JOIN?

Answer: Some employees, such as Branch Managers, may not have a manager. An INNER JOIN would remove them from the result.

Q6.Explain COUNT(*), COUNT(interest_rate), COUNT(DISTINCT account_type) and AVG(balance).

Answer: Count: Yes, counts every rows, value on account is 36
Count(interest_rate): no, skips NULLs, 36(no NULLs in this column)
Count(DISTINCT account_type): No, 3-savings,current,salary
AVG(balance): No, divides by non-null count, 220372.35

Q7.State the GROUP BY rule.

Answer: Every selected column must either be in GROUP BY or be inside an aggregate function. The database cannot randomly choose a value when a group contains different values.

Q8.Complete the WHERE/HAVING table.

Answer: WHERE filters individual rows and runs before grouping. HAVING filters groups and runs after grouping. HAVING can use aggregate functions. Use WHERE for row filtering and HAVING for aggregate filtering.

Q9.Why does NOT IN return zero rows and give two fixes.

Answer: The subquery may contain NULL. With NOT IN, a NULL can make the comparison unknown. Fix it by removing NULLs or by using NOT EXISTS.

```
SELECT COUNT(*)
FROM customer c
WHERE c.first_name NOT IN (
  SELECT nominee_name
  FROM fixed_deposit
  WHERE nominee_name IS NOT NULL
);
```

```
SELECT COUNT(*)
FROM customer c
WHERE NOT EXISTS (
  SELECT 1
```

```

FROM fixed_deposit f
WHERE f.nominee_name = c.first_name
);

```

Q10. Explain fan-out.

Answer: Fan-out happens when one row matches many rows in another table. For example, customer → account → transactions can repeat the same customer, making counts or totals look higher than they really are.

Level 2 – Applying SQL

Q11. List every card with account number, customer name, branch, card type, masked number and status.

```

SELECT
c.card_type,
c.masked_number,
c.status,
a.account_number, CONCAT(cu.first_name, ' ',
cu.last_name) AS customer_name,
b.branch_name
FROM card c
JOIN account a ON a.account_id = c.account_id
JOIN customer cu ON cu.customer_id = a.customer_id
JOIN branch b ON b.branch_id = a.branch_id
ORDER BY b.branch_name, customer_name;

```

Q12. List customers at Koramangala and Whitefield with accounts and balances, including customers with no account.

```

SELECT cu.customer_id, CONCAT(cu.first_name, ' ',
cu.last_name) AS customer_name,
a.account_number,
a.balance
FROM
customer cu
JOIN branch b ON b.branch_id = cu.home_branch_id
LEFT JOIN account a ON a.customer_id = cu.customer_id
WHERE b.branch_name IN ('Koramangala', 'Whitefield')
ORDER BY a.balance DESC NULLS LAST;

```

Q13. Find accounts that have never had a transaction.

```

SELECT
a.account_number,
a.account_type,
a.status, CONCAT(c.first_name, ' ', c.last_name) AS
customer_name
FROM account a
JOIN customer c ON c.customer_id = a.customer_id
LEFT JOIN account_txn t ON t.account_id = a.account_id
WHERE t.account_id IS NULL;

```

Q14. Find accounts with no card issued. Give count and list.

```

SELECT
a.account_number,
a.account_type, CONCAT(c.first_name, ' ',
c.last_name) AS customer_name
FROM account a
JOIN customer c ON c.customer_id = a.customer_id
LEFT JOIN card ca ON ca.account_id = a.account_id

```

```

WHERE ca.account_id IS NULL;
SELECT COUNT(*)
FROM account a
LEFT JOIN card ca ON ca.account_id = a.account_id
WHERE ca.account_id IS NULL;

```

Q15. Produce the full employee hierarchy.

```

SELECT
e.employee_name,
e.designation,
b.branch_name, COALESCE(m.employee_name,
'Branch Manager') AS manager_name
FROM employee e
JOIN branch b ON b.branch_id = e.branch_id
LEFT JOIN employee m ON m.employee_id = e.manager_id
ORDER BY b.branch_name, e.employee_name;

```

Q16. For each Branch Manager, show team size and team salary excluding their own.

```

SELECT
m.employee_name AS manager_name,
b.branch_name,
COUNT(e.employee_id) AS
team_size,
COALESCE(SUM(e.salary), 0) AS team_salary
FROM employee m
JOIN branch b ON b.branch_id = m.branch_id
LEFT JOIN employee e ON e.manager_id = m.employee_id
WHERE m.designation = 'Branch Manager'
GROUP BY m.employee_id, m.employee_name, b.branch_name
ORDER BY b.branch_name;

```

Q17. Build loan risk bands and report loan count and total outstanding using a non-equi join.

```

WITH loan_risk_band AS (
SELECT 'L1' AS band_code, 'Small' AS band_name, 0 AS min_outstanding, 500000 AS max_outstanding
UNION ALL SELECT 'L2', 'Medium', 500000, 2000000
UNION ALL SELECT 'L3', 'Large', 2000000, 5000000
UNION ALL SELECT 'L4', 'Very Large', 5000000, 99999999
)
SELECT
r.band_code,
r.band_name,
COUNT(l.loan_id) AS
loan_count,
COALESCE(SUM(l.outstanding), 0) AS total_outstanding
FROM loan_risk_band r
LEFT JOIN loan l
ON l.outstanding >= r.min_outstanding
AND l.outstanding < r.max_outstanding
GROUP BY r.band_code, r.band_name
ORDER BY r.band_code;

```

Q18. For each branch, report customers, accounts, total balance and average balance.

```

SELECT
b.branch_name, COUNT(DISTINCT
c.customer_id) AS customer_count,
COUNT(DISTINCT a.account_id) AS account_count,
COALESCE(SUM(a.balance), 0) AS total_balance,

```

```

AVG(a.balance) AS average_balance
FROM branch b
LEFT JOIN customer c ON c.home_branch_id = b.branch_id
LEFT JOIN account a ON a.customer_id = c.customer_id
GROUP BY b.branch_id, b.branch_name
ORDER BY total_balance DESC;

```

Q19. For each loan type, show count, principal, outstanding and outstanding percentage.

```

SELECT loan_type,
COUNT(*) AS
loan_count,
SUM(principal) AS total_principal,
SUM(outstanding) AS total_outstanding,
ROUND(SUM(outstanding) * 100.0 / NULLIF(SUM(principal), 0), 1)
AS outstanding_percentage
FROM loan
GROUP BY loan_type;

```

Q20. For each channel, show successful/failed transactions, successful value and failure rate.

```

SELECT channel, COUNT(*) FILTER (WHERE status =
'SUCCESS') AS successful_transactions,
COUNT(*) FILTER (WHERE status = 'FAILED') AS failed_transactions,
COALESCE(SUM(amount) FILTER (WHERE status = 'SUCCESS'), 0) AS successful_value,
ROUND(
COUNT(*) FILTER (WHERE status = 'FAILED') * 100.0 /
NULLIF(COUNT(*), 0), 2
) AS failure_rate
FROM account_txn
GROUP BY channel
ORDER BY failure_rate DESC;

```

Q21. Find branches whose total outstanding loan book exceeds 2,000,000.

```

SELECT
b.branch_name,
COUNT(l.loan_id) AS
loan_count,
SUM(l.outstanding) AS total_outstanding
FROM branch b
JOIN loan l ON l.branch_id = b.branch_id
GROUP BY b.branch_id, b.branch_name
HAVING SUM(l.outstanding) > 2000000;

```

Q22. Find customers with at least two accounts.

```

SELECT
CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
COUNT(a.account_id) AS account_count,
SUM(a.balance) AS combined_balance
FROM customer c
JOIN account a ON a.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(a.account_id) >= 2
ORDER BY combined_balance DESC;

```

Q23. Produce a monthly transaction summary.

```

SELECT
DATE_TRUNC('month', txn_date) AS month,
COUNT(*) AS transaction_count,

```

```

SUM(CASE WHEN txn_type = 'CREDIT' THEN amount ELSE 0 END) AS
total_credits, SUM(CASE WHEN txn_type = 'DEBIT' THEN amount ELSE 0
END) AS total_debits,
SUM(CASE
WHEN txn_type = 'CREDIT' THEN amount
WHEN txn_type = 'DEBIT' THEN -amount
ELSE 0
END) AS net_movement
FROM account_txn
GROUP BY DATE_TRUNC('month', txn_date)
ORDER BY month;

```

Q24. List accounts whose balance is more than twice the bank-wide average.

```

SELECT account_number, balance
FROM account
WHERE balance > 2 * (
SELECT AVG(balance)
FROM account
);

```

Q25. For each branch, show the balance of its largest account using a correlated subquery.

```

SELECT
b.branch_name, (
SELECT MAX(a2.balance)
FROM account a2
JOIN customer c2 ON c2.customer_id = a2.customer_id
WHERE c2.home_branch_id = b.branch_id
) AS largest_balance
FROM branch b;

```

Q26. Find customers who hold a loan but no fixed deposit, using NOT EXISTS and LEFT JOIN.

```

SELECT c.customer_id, c.first_name, c.last_name
FROM customer c
WHERE EXISTS (
SELECT 1 FROM loan l
WHERE l.customer_id = c.customer_id
)
AND NOT EXISTS (
SELECT 1 FROM fixed_deposit f
WHERE f.customer_id = c.customer_id
);

SELECT DISTINCT c.customer_id, c.first_name, c.last_name
FROM customer c
JOIN loan l ON l.customer_id = c.customer_id
LEFT JOIN fixed_deposit f ON f.customer_id = c.customer_id
WHERE f.customer_id IS NULL;

```

Q27. Find customers whose quarter transaction value is above the average total.

```

WITH customer_totals AS (
SELECT
c.customer_id, CONCAT(c.first_name, ' ',
c.last_name) AS customer_name,
COALESCE(SUM(t.amount), 0) AS total_value
FROM customer c
LEFT JOIN account a ON a.customer_id = c.customer_id
LEFT JOIN account_txn t ON t.account_id = a.account_id

```

```

WHERE t.txn_date >= DATE_TRUNC('quarter', CURRENT_DATE)
GROUP BY c.customer_id, c.first_name, c.last_name
)
SELECT *
FROM customer_totals
WHERE total_value > (
SELECT AVG(total_value)
FROM customer_totals
);

```

Q28. Produce one list of every product holding.

```

SELECT customer_id, 'FIXED_DEPOSIT' AS product_type, amount
FROM fixed_deposit
UNION ALL
SELECT customer_id, 'LOAN' AS product_type, principal AS amount
FROM loan
UNION ALL
SELECT customer_id, 'ACCOUNT' AS product_type, balance AS amount
FROM account
ORDER BY customer_id, product_type;

```

Q29. (Core) Using set operators, find both-product, deposit-only and loan-only customers.

```

SELECT customer_id FROM fixed_deposit
INTERSECT
SELECT customer_id FROM loan;

SELECT customer_id FROM fixed_deposit
EXCEPT
SELECT customer_id FROM loan;

SELECT customer_id FROM loan
EXCEPT
SELECT customer_id FROM fixed_deposit;

```

Level 3 – Analysis and Decisions

Q30. Identify defects in the monthly pack query and rewrite it correctly.

Answer: The main defect is fan-out: joining accounts and transactions can repeat account balances and customer rows. Also, an inclusive end date can be risky when timestamps contain time. Aggregate transactions before joining them to branch/customer totals.

```

WITH txn_totals AS (
SELECT
a.customer_id, SUM(t.amount)
AS total_transacted
FROM account a
JOIN account_txn t ON t.account_id = a.account_id
WHERE t.txn_date >= DATE '2026-01-01'
AND t.txn_date < DATE '2026-04-01'
GROUP BY a.customer_id
)
SELE
CT
b.branch_name, COUNT(DISTINCT
c.customer_id) AS customers,
COALESCE(SUM(a.balance), 0) AS total_deposits,
COALESCE(SUM(tt.total_transacted), 0) AS total_transacted
FROM branch b
JOIN customer c ON c.home_branch_id = b.branch_id
LEFT JOIN account a ON a.customer_id = c.customer_id

```

```
LEFT JOIN txn_totals tt ON tt.customer_id = c.customer_id
GROUP BY b.branch_id, b.branch_name;
```

Q31. Explain why GROUP BY/HAVING alone cannot find the top three customers in each branch.

Answer: GROUP BY can calculate branch totals, but it cannot directly rank rows within each branch. A window function such as ROW_NUMBER() or RANK(), with PARTITION BY branch, is needed.

Q32.What does a NOT IN report returning zero rows for six weeks tell you?

Answer: NULL handling needs to be checked carefully. NOT IN can fail when the subquery contains NULL. Reports should have checks for unexpected zero results. A zero compliance exception list should be verified instead of automatically assumed to be correct.

Q33. The report takes 40 seconds on 40 million transactions. Give three actions without changing SQL.

Answer: 1. Check or add useful indexes on join/filter columns.
2. Check the execution plan to find the slow operation.
3. Review scheduling and resource usage so the report does not compete with the branch-opening workload.

Q34. Prove that every acquired-bank transaction landed in ICICI and that ICICI gained none.

Answer: A non-empty first result means an acquired transaction is missing in ICICI. A non-empty second result means ICICI has a transaction not present in the extract. A difference can also come from formatting, spaces, date/time representation or another non-business-key difference.

```
SELECT *
FROM acquired_transactions
EXCEPT
SELECT *
FROM icici_transactions;
```

```
SELECT *
FROM icici_transactions
EXCEPT
SELECT *
FROM acquired_transactions;
```

Q35.Reply to ticket ICICI-4530.

Answer: The updated pack now gives a branch-wise view of customers, accounts, balances, loans and transactions. It also shows transaction success and failure rates by channel. One finding is that some accounts have never had a transaction. Another finding is that some branches have loan outstanding amounts above 20 lakh. The report also identifies customers with multiple accounts. Monthly transaction totals help show credit and debit movement. Branches can be compared using these measures. However, joins involving transactions can create duplicate totals if they are not aggregated correctly. Therefore, totals should be checked before important decisions are made. The report also depends on the accuracy and completeness of the source data.